

Question 1: Debugging and Optimizing K-Means Clustering

Introduction

K-Means is an unsupervised machine learning algorithm used to group similar data points into **K clusters** based on their features. It works by selecting initial centroids, assigning each data point to the nearest centroid, and repeatedly updating the centroid positions until convergence. K-Means is widely used in customer segmentation, image compression, recommendation systems, and pattern recognition.

However, K-Means may produce **different cluster assignments for similar datasets**. This inconsistency occurs because the algorithm is sensitive to centroid initialization, the choice of the number of clusters, outliers, and feature scaling. To improve clustering performance, these issues must be identified through debugging and resolved using appropriate optimization techniques.

Problem Analysis

Suppose we have a customer dataset containing the following features:

- Age
- Annual Income
- Spending Score

When the K-Means algorithm is executed multiple times on this dataset, the resulting clusters are different in each run. This indicates that the clustering process is unstable and needs debugging.

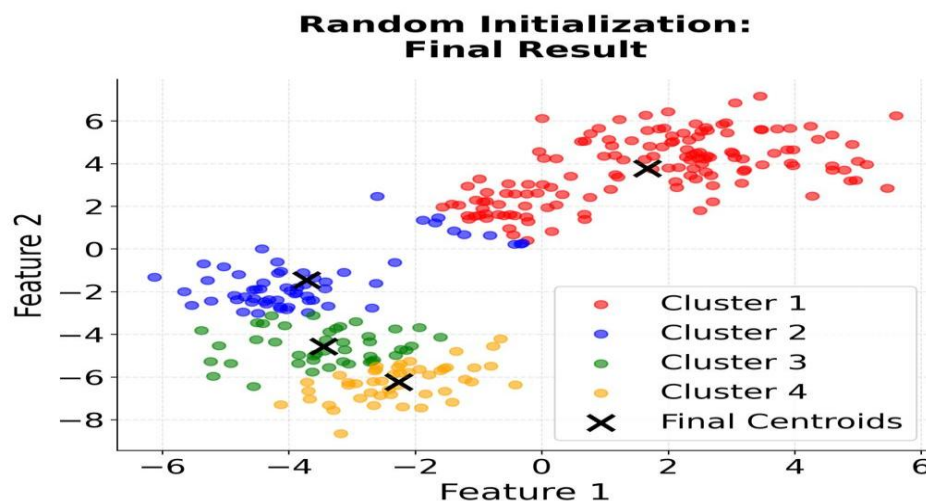


Figure 1: Initial K-Means clustering showing unstable/random cluster assignments.

Possible Causes of Inconsistent Clustering

1. Poor Initialization of Centroids

K-Means randomly selects the initial centroids. Different starting points may lead to different final clusters because the algorithm converges to a local optimum instead of the global optimum.

Effect:

- Different results for every execution.
 - Unstable cluster assignments.
-

2. Incorrect Number of Clusters (K)

Choosing an inappropriate value for **K** affects clustering quality.

- Too few clusters combine different groups.
- Too many clusters split natural groups unnecessarily.

This results in poor clustering accuracy.

3. Sensitivity to Outliers

Outliers are data points that lie far away from the majority of observations.

Since K-Means calculates centroids using the **mean**, extreme values pull the centroid away from the actual cluster center.

Effect:

- Incorrect centroid positions.
 - Poor cluster formation.
-

4. Different Feature Scales

Suppose the dataset contains:

Feature	Range
Age	18–60
Annual Income	10,000–250,000

Because income has much larger values, it dominates the distance calculation, making Age almost insignificant.

5. Overlapping Clusters

If the actual groups in the dataset overlap significantly, K-Means cannot clearly separate them, resulting in inconsistent clustering.

Debugging Approaches

After identifying the possible causes, the next step is to debug the model and determine the exact reason for the inconsistent clusters.

Debugging Step 1: Run K-Means Multiple Times

Run the algorithm repeatedly to check whether the cluster assignments change.

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
print(kmeans.labels_)
```

Observation

- If the labels change every time the program is executed, random initialization is affecting the clustering process.

Purpose:

To determine whether the inconsistency is caused by randomly selected initial centroids.

Debugging Step 2: Visualize the Clusters

Visualize the clusters and centroid positions using a scatter plot.

```
import matplotlib.pyplot as plt
plt.scatter(X[:,0], X[:,1], c=kmeans.labels_)
plt.scatter(kmeans.cluster_centers_[:,0],
            kmeans.cluster_centers_[:,1],
            color='red',
            marker='X',
            s=200)
plt.title("K-Means Clustering")
plt.show()
```

Observation

- If the centroid positions vary significantly after each execution, the clustering is unstable.

Purpose:

Visualization helps identify whether clusters are well separated and whether centroids are placed correctly.

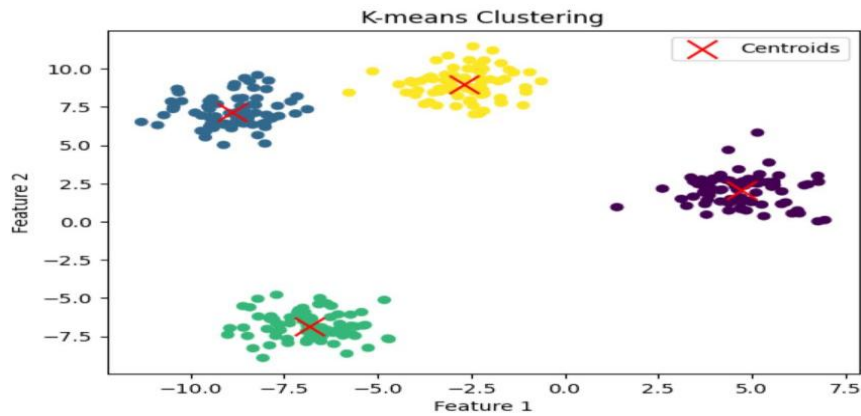


Figure 2: Scatter plot showing clustered data points with centroids (red X) after applying the K-Means algorithm.

Debugging Step 3: Detect Outliers

Outliers can be identified using a box plot.

```
import seaborn as sns
sns.boxplot(data=df)
```

Observation

If extreme values are present, they may influence centroid locations and reduce clustering accuracy.

Purpose:

To identify abnormal values before applying K-Means.

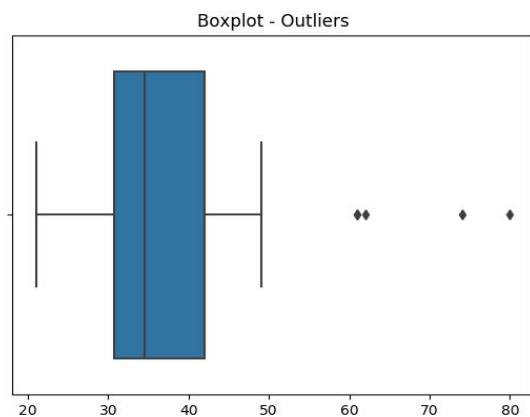


Figure 3: Box plot used to identify outliers in the dataset before clustering.

Debugging Step 4: Check Feature Scaling

Examine the statistical summary of the dataset.

```
print(df.describe())
```

Observation

If one feature has values much larger than the others, it dominates the Euclidean distance calculation.

Purpose:

To determine whether normalization is required.

Optimization Strategies

Once the problems are identified, the following optimization techniques can be applied.

1. Use K-Means++ Initialization

Instead of random centroid selection, K-Means++ chooses well-separated initial centroids.

```
kmeans = KMeans(  
    n_clusters=3,  
    init='k-means++'  
)
```

Advantages

- Better initial centroids
 - Faster convergence
 - More consistent clustering results
 - Reduced chances of local optimum
-

2. Normalize the Dataset

Normalize all features before clustering.

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Advantages

- Every feature contributes equally.
 - Prevents large-valued features from dominating the distance calculation.
-

3. Find the Optimal Number of Clusters (Elbow Method)

The Elbow Method helps determine the most suitable value of **K**.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
inertia = []

for k in range(1,11):
    model = KMeans(n_clusters=k)
    model.fit(X_scaled)
    inertia.append(model.inertia_)

plt.plot(range(1,11), inertia, marker='o')
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

Interpretation

The point where the graph forms an "elbow" indicates the optimal number of clusters.

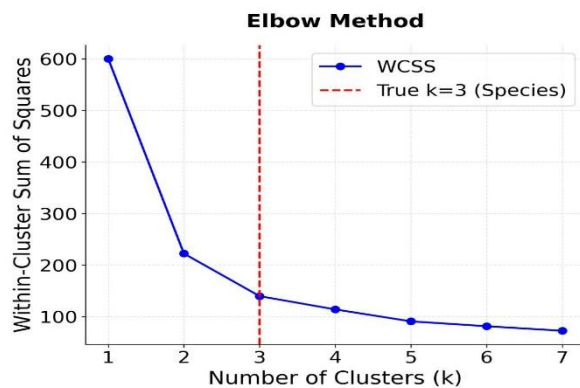


Figure 4: Elbow Method graph used to determine the optimal value of **K**.

4. Evaluate Cluster Quality Using Silhouette Analysis

The Silhouette Score measures how well the clusters are separated.

```
from sklearn.metrics import silhouette_score
score = silhouette_score(
    X_scaled,
    kmeans.fit_predict(X_scaled)
)
print("Silhouette Score:", score)
```

Interpretation

- Score close to 1 → Excellent clustering
- Score close to 0 → Overlapping clusters

- **Score less than 0** → Poor clustering

A higher silhouette score indicates better cluster quality.

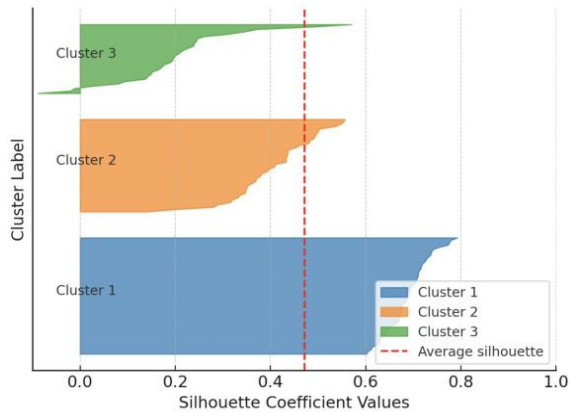


Figure 5: Silhouette Analysis used to evaluate the quality of clustering.

5. Remove Outliers Before Clustering

Outliers can be removed using statistical methods such as:

- Interquartile Range (IQR)
- Z-score Method

Removing outliers improves centroid placement and increases clustering stability.

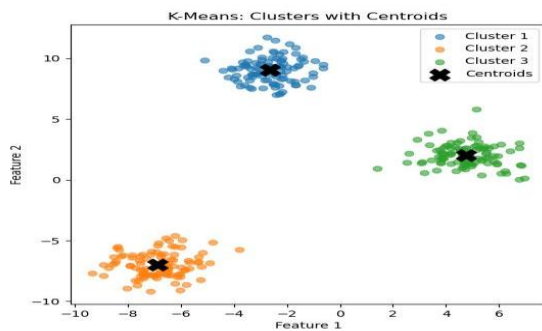


Figure 6: Final clustering result after applying K-Means++, normalization, and selecting the optimal number of clusters.

Expected Result After Optimization

Before Optimization	After Optimization
Random cluster assignments	Stable cluster assignments
Poor centroid initialization	Better centroids using K-Means++
Incorrect number of clusters	Optimal K selected using Elbow Method
Dominance of large-scale features	Balanced features after normalization
Low Silhouette Score	Higher Silhouette Score
Sensitive to outliers	Improved clustering after outlier removal

Conclusion

K-Means clustering may produce inconsistent results due to **random initialization, incorrect selection of the number of clusters, outliers, and differences in feature scales**. These problems can be identified through debugging techniques such as repeated execution, cluster visualization, outlier detection, and checking feature statistics. The clustering performance can be significantly improved by using **K-Means++ initialization, feature normalization, Elbow Method, Silhouette Analysis, and outlier removal**. These optimization techniques produce more stable, accurate, and meaningful clusters, making the K-Means algorithm more reliable for real-world applications.

Question 2: Debugging and Optimizing a Support Vector Machine (SVM) Model

Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression problems. It works by finding the optimal hyperplane that best separates different classes while maximizing the margin between them. SVM is widely used in image classification, text classification, face recognition, handwriting recognition, and medical diagnosis.

Although SVM performs well on many datasets, it may underperform when applied to nonlinear datasets if inappropriate kernels, poor hyperparameter settings, or improper feature scaling are used. Therefore, debugging and optimization techniques are essential to improve the model's performance.

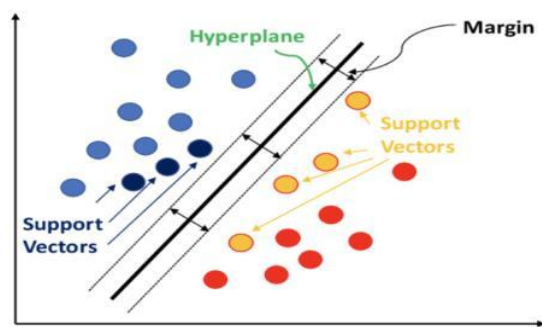


Figure 1: Basic working of the Support Vector Machine (SVM) classifier showing the optimal hyperplane, support vectors, and maximum margin.

Problem Analysis

Consider a nonlinear dataset where the classes cannot be separated using a straight line. If an SVM model is trained using incorrect settings, it may produce poor classification accuracy, high misclassification rates, and weak decision boundaries.

The objective is to identify the possible causes of poor performance, debug the model, and optimize it using appropriate machine learning techniques.

Possible Causes of Poor Performance

1. Incorrect Kernel Selection

The kernel function determines how the SVM separates the data.

Common kernels include:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel
- Sigmoid Kernel

If a **Linear Kernel** is used for nonlinear data, the model cannot separate the classes effectively.

Effect

- Low accuracy
 - Incorrect decision boundary
 - Underfitting
-

2. Improper Hyperparameter Tuning

Important hyperparameters include:

C (Regularization Parameter)

Controls the trade-off between maximizing the margin and minimizing classification errors.

- High C → Less tolerance for errors, possible overfitting.
- Low C → Larger margin, possible underfitting.

Gamma

Controls how much influence each training sample has.

- High Gamma → Complex decision boundary, overfitting.
 - Low Gamma → Smooth decision boundary, underfitting.
-

3. Lack of Feature Scaling

SVM relies on distance calculations.

Suppose a dataset contains:

Feature	Range
Age	18–60
Salary	20,000–150,000

Salary dominates the distance calculation, making Age almost insignificant.

4. Noisy Data

Incorrect labels or noisy observations confuse the classifier and reduce prediction accuracy.

5. Class Imbalance

If one class contains significantly more samples than another, the classifier becomes biased toward the majority class.

Debugging Approaches

The following steps help identify why the SVM model is underperforming.

Debugging Step 1: Train a Basic SVM Model

First, train a simple SVM using the default parameters.

```
from sklearn.svm import SVC
model = SVC()
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Observation

If the accuracy is low, the default model is not suitable for the dataset.

Purpose:

To establish a baseline model before optimization.

Debugging Step 2: Compare Different Kernel Functions

Test multiple kernels and compare their accuracy.

```
from sklearn.svm import SVC
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
for kernel in kernels:
    model = SVC(kernel=kernel)
    model.fit(X_train, y_train)
    accuracy = model.score(X_test, y_test)
    print(kernel, accuracy)
```

Observation

The kernel with the highest accuracy is the most appropriate for the dataset.

Purpose:

To determine whether poor performance is caused by an incorrect kernel.

Debugging Step 3: Check Model Performance Using a Confusion Matrix

```
from sklearn.metrics import confusion_matrix
prediction = model.predict(X_test)
cm = confusion_matrix(y_test, prediction)
print(cm)
```

Observation

The confusion matrix identifies:

- True Positives
- True Negatives
- False Positives
- False Negatives

Purpose:

To determine where the classifier is making mistakes.

Debugging Step 4: Generate Classification Report

```
from sklearn.metrics import classification_report  
print(classification_report(y_test,prediction))
```

Observation

The report provides:

- Precision
- Recall
- F1 Score
- Accuracy

Purpose:

To analyze the overall quality of the classifier.

Debugging Step 5: Check Feature Scaling

Display the statistical summary.

```
print(df.describe())
```

If one feature has much larger values than others, scaling is necessary.

Purpose:

To identify whether feature scaling is affecting SVM performance.

Optimization Strategies

After identifying the problems, the following optimization techniques improve model performance.

Optimization Strategy 1: Apply Feature Scaling

Normalize the dataset before training.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Advantages

- Equal contribution of every feature.
- Better distance calculations.
- Improved prediction accuracy.

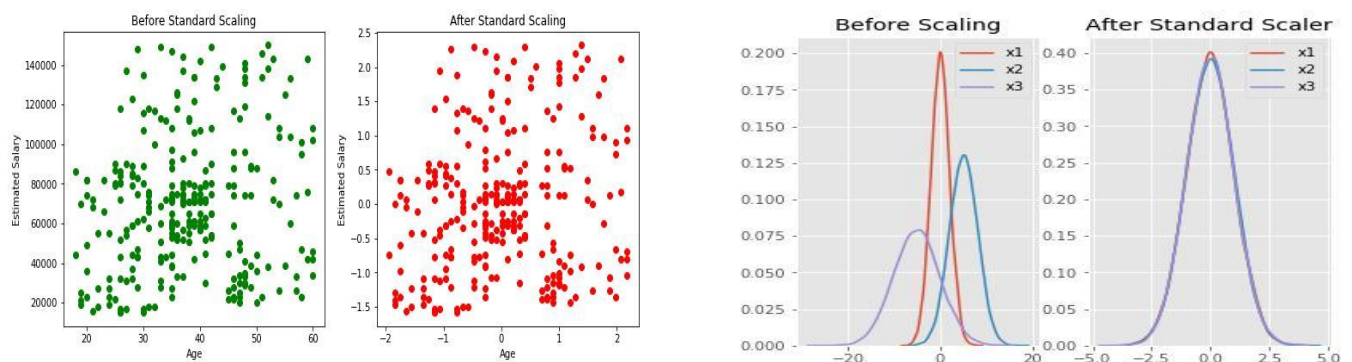


Figure 3: Dataset before and after feature scaling using StandardScaler.

Optimization Strategy 2: Use the RBF Kernel

For nonlinear datasets, RBF is usually the best choice.

```
from sklearn.svm import SVC
model = SVC(kernel='rbf')
model.fit(X_train_scaled, y_train)
prediction = model.predict(X_test_scaled)
```

Advantages

- Handles nonlinear data efficiently.
- Produces flexible decision boundaries.
- Improves classification performance.

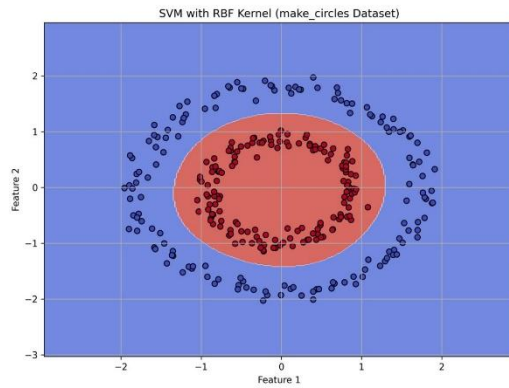


Figure 4: Decision boundary created using the Radial Basis Function (RBF) kernel for nonlinear classification.

Optimization Strategy 3: Hyperparameter Tuning Using Grid Search

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
parameters = {
    'C':[0.1,1,10,100],
    'gamma':[1,0.1,0.01,0.001],
    'kernel':['rbf','poly']
}
```

```
grid = GridSearchCV(SVC(),parameters)
grid.fit(X_train_scaled,y_train)
print(grid.best_params_)
```

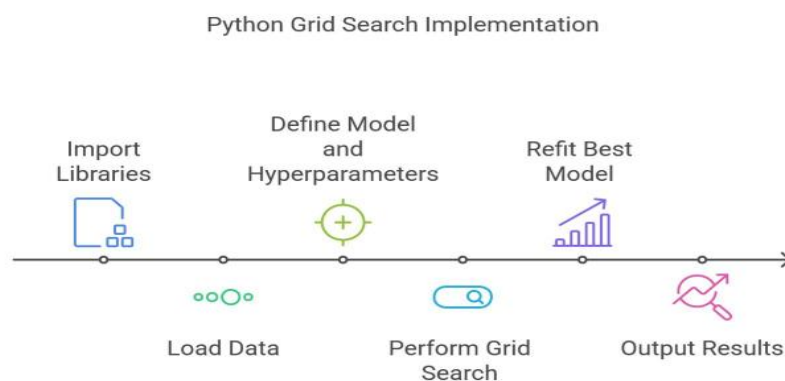


Figure 5: Hyperparameter tuning using Grid Search to determine the optimal values of C, Gamma, and Kernel.

Advantages

- Finds the best values for C, Gamma, and Kernel.
 - Improves model accuracy.
 - Reduces overfitting.
-

Optimization Strategy 4: Perform Cross Validation

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model,
                          X_train_scaled,
                          y_train,
                          cv=5)
print(scores)
print(scores.mean())
```

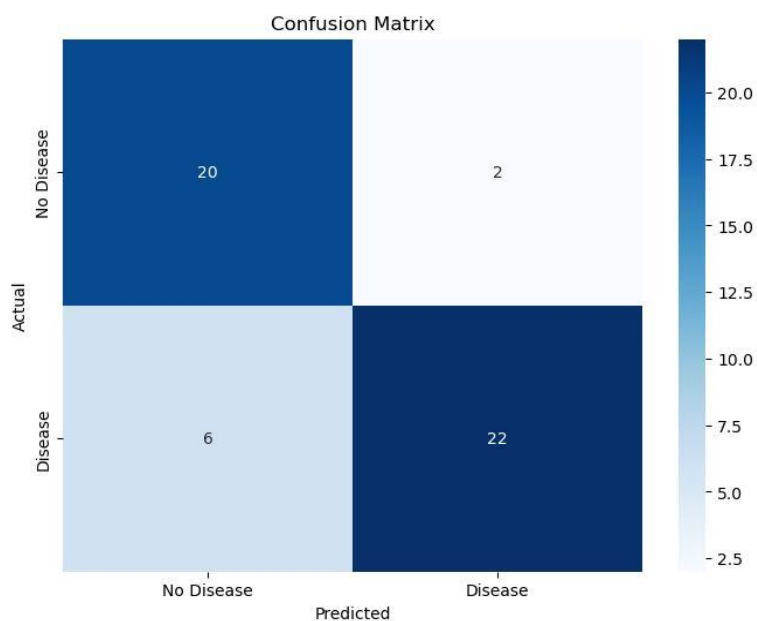


Figure 6: Confusion Matrix showing the performance of the SVM classifier by comparing predicted and actual class labels.

Advantages

- Evaluates model stability.
 - Reduces bias caused by a single train-test split.
 - Produces more reliable performance estimates.
-

Optimization Strategy 5: Handle Class Imbalance

If the dataset is imbalanced, assign balanced class weights.

```
model = SVC(class_weight='balanced')
model.fit(X_train_scaled,
          y_train)
```

Alternative methods include:

- SMOTE
- Random Oversampling
- Random Undersampling

These techniques improve classification performance on minority classes.

Expected Results After Optimization

Before Optimization	After Optimization
Low Accuracy	Higher Accuracy
Incorrect Kernel	Appropriate Kernel Selected
Unscaled Features	Standardized Features
Poor Decision Boundary	Better Decision Boundary
Underfitting / Overfitting	Balanced Model
Low Precision & Recall	Improved Precision & Recall

Conclusion

Support Vector Machine models may underperform due to incorrect kernel selection, poor hyperparameter tuning, lack of feature scaling, noisy data, and class imbalance. These issues can be identified through debugging techniques such as training a baseline model, comparing different kernels, analyzing the confusion matrix, generating a classification report, and checking feature scaling. The model's performance can then be significantly improved by applying **feature scaling, selecting the appropriate kernel (such as RBF), tuning hyperparameters using Grid Search, performing cross-validation, and handling class imbalance**. These optimization strategies lead to higher accuracy, better generalization, and a more reliable SVM model for nonlinear classification problems.

Question 3: Debugging and Optimizing a Recurrent Neural Network (RNN) for Natural Language Processing (NLP)

Introduction

Recurrent Neural Networks (RNNs) are a type of deep learning algorithm specifically designed to process sequential data such as text, speech, and time-series data. Unlike traditional neural networks, RNNs use feedback connections that allow information from previous time steps to influence future predictions. Due to this capability, RNNs are widely used in Natural Language Processing (NLP) tasks such as language translation, text generation, sentiment analysis, speech recognition, and chatbot development.

However, standard RNNs often struggle to learn long-term dependencies in long sequences. This results in poor prediction accuracy and unstable training. Therefore, debugging techniques and optimization strategies are required to improve the model's performance.

Problem Analysis

Consider an NLP model that uses a Recurrent Neural Network (RNN) to classify movie reviews as positive or negative. When the input sentences become longer, the model starts forgetting important words that appeared earlier in the sequence. As a result, the prediction accuracy decreases significantly.

This problem mainly occurs because the RNN cannot effectively retain long-term information during training.

Possible Causes of Poor Performance

1. Vanishing Gradient Problem

During backpropagation, gradients become extremely small as they pass through many hidden layers.

As a result:

- Early layers receive almost no updates.
 - The network fails to learn long-term dependencies.
 - Important information is forgotten.
-

2. Exploding Gradient Problem

Sometimes gradients become extremely large during training.

Effects include:

- Unstable learning
 - Oscillating loss values
 - Failure of the model to converge
-

3. Limited Memory Capacity

A standard RNN stores information only in a hidden state.

When sequences become longer:

- Older information disappears.
 - The model remembers only recent words.
-

4. Long Input Sequences

Very long sentences increase computational complexity and make it difficult for standard RNNs to capture meaningful context.

5. Poor Hyperparameter Selection

Incorrect hyperparameters such as:

- High learning rate
- Small hidden layer size
- Insufficient training epochs
- Small batch size

can reduce the model's learning capability.

Debugging Approaches

The following debugging techniques help identify why the RNN model performs poorly.

Debugging Step 1: Build and Train a Basic RNN Model

Train a simple RNN model using TensorFlow/Keras.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
model = Sequential()
model.add(SimpleRNN(64,
                    input_shape=(100,1)))

model.add(Dense(1,
                activation='sigmoid'))
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.summary()
```

Observation

This creates a basic RNN model.

Purpose:

To establish a baseline model before optimization.

Debugging Step 2: Monitor Training Loss

Train the model and observe training performance.

```
history = model.fit(
    X_train,
    y_train,
    epochs=10,
    validation_data=(X_test,y_test)
)
```

Observation

If training loss decreases very slowly or stops improving, the model may be suffering from vanishing gradients.

Purpose:

To determine whether the network is learning effectively.

Debugging Step 3: Evaluate Model Accuracy

```
loss,accuracy = model.evaluate(
    X_test,
    y_test
)
print("Accuracy =",accuracy)
```

Observation

Low accuracy indicates that the RNN is unable to capture long-term dependencies.

Debugging Step 4: Plot Training and Validation Accuracy

```
import matplotlib.pyplot as plt
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend(["Training",
           "Validation"])
plt.show()
```

Observation

Large differences between training and validation accuracy indicate overfitting.

Purpose:

To evaluate model performance visually.

Debugging Step 5: Display Model Architecture

```
model.summary()
```

Observation

The model summary shows:

- Number of layers
- Number of parameters
- Output shapes

Purpose:

To verify that the network architecture is correctly designed.

Optimization Strategies

After identifying the problems, the following optimization techniques improve the model.

Optimization Strategy 1: Replace RNN with LSTM

LSTM networks contain memory cells that preserve long-term information.

```
from tensorflow.keras.layers import LSTM
model = Sequential()
model.add(LSTM(64,
               input_shape=(100,1)))
model.add(Dense(1,
               activation='sigmoid'))
```

Advantages

- Learns long-term dependencies.
 - Reduces vanishing gradients.
 - Improves prediction accuracy.
-

Optimization Strategy 2: Replace RNN with GRU

GRU is a simplified version of LSTM.

```
from tensorflow.keras.layers import GRU
model = Sequential()
```

```
model.add(GRU(64,  
              input_shape=(100,1)))  
model.add(Dense(1,  
                activation='sigmoid'))
```

Advantages

- Faster training.
 - Fewer parameters.
 - Better memory retention.
-

Optimization Strategy 3: Use Dropout

Dropout prevents overfitting.

```
from tensorflow.keras.layers import Dropout  
model = Sequential()  
model.add(LSTM(64,  
              input_shape=(100,1)))  
model.add(Dropout(0.3))  
model.add(Dense(1,  
                activation='sigmoid'))
```

Advantages

- Reduces overfitting.
 - Improves generalization.
-

Optimization Strategy 4: Gradient Clipping

Gradient clipping prevents exploding gradients.

```
from tensorflow.keras.optimizers import Adam  
  
optimizer = Adam(  
    learning_rate=0.001,  
    clipnorm=1.0  
)  
  
model.compile(  
    optimizer=optimizer,  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

Advantages

- Stable training.
 - Prevents exploding gradients.
-

Optimization Strategy 5: Use Early Stopping

Stop training when validation performance no longer improves.

```
from tensorflow.keras.callbacks import EarlyStopping
early = EarlyStopping(
    monitor='val_loss',
    patience=3
)
model.fit(
    X_train,
    y_train,
    epochs=20,
    callbacks=[early]
)
```

Advantages

- Prevents overfitting.
 - Saves training time.
-

Optimization Strategy 6: Use Attention Mechanism

Modern NLP models use Attention to focus on important words in a sentence.

Although TensorFlow provides advanced attention layers, conceptually it can be added as follows:

```
from tensorflow.keras.layers import Attention
attention = Attention()
```

Advantages

- Focuses on relevant words.
 - Improves context understanding.
 - Increases NLP model performance.
-

Expected Results After Optimization

Before Optimization	After Optimization
Poor long-term memory	Better long-term dependency learning
Vanishing gradients	Reduced vanishing gradients using LSTM/GRU
Unstable training	Stable training using Gradient Clipping
Overfitting	Reduced overfitting using Dropout and Early Stopping
Lower Accuracy	Higher Accuracy
Standard RNN	Improved LSTM/GRU with Attention

Conclusion

Recurrent Neural Networks often struggle to learn long-term dependencies because of vanishing gradients, exploding gradients, limited memory capacity, and poor hyperparameter selection. These problems can be identified through debugging techniques such as monitoring training loss, evaluating accuracy, plotting learning curves, and inspecting the model architecture. The model performance can be significantly improved by replacing standard RNNs with **LSTM or GRU networks**, applying **Dropout**, using **Gradient Clipping**, implementing **Early Stopping**, and incorporating **Attention Mechanisms**. These optimization strategies improve memory retention, training stability, and overall accuracy, making RNN-based NLP models more effective for real-world applications.